F0003 CA3 · AGENTIC AI · PROJECT SYNOPSIS

Skopeo: multi-agent repository intelligence and risk analysis

Team	Kushagra Goyal (GitHub `mardromus`) · Mayank Hete (GitHub `Rancidcake`) · Abhilaksh Saini
Institute	Department of Computer Science and Engineering, Symbiosis Institute of Technology, Pune
Repository	https://github.com/mardromus/Skopeo
Framework	LangGraph (Python 3.12+, FastAPI) · React UI · Groq-hosted LLMs (openai/gpt-oss-120b, openai/gpt-oss-20b)

Problem

Before a team adopts, upgrades or ships a codebase, someone has to decide what in it is actually risky. Today that means running separate tools — vulnerability scanners, linters, coverage, license checkers — whose results are siloed and noisy. No tool says that the vulnerable library sits in the login path, that upgrading it breaks two call sites, and that nothing tests that code; and many alerts are false, such as an "exposed AWS key" that is the documented example value in a test fixture. Engineers end up either drowning in alerts or missing the compound risks that matter.

Solution

Skopeo investigates a GitHub repository the way a small review team would. An **Orchestrator agent** reads the repository and plans which of **eight specialist agents** (security, dependency health, code quality, API compatibility, test reliability, license, maintenance, performance) to run; they work in parallel and post evidence to a shared **Evidence Store** (blackboard). When evidence raises new questions, agents request follow-up work and the orchestrator re-plans; if an agent fails, it retries with a degraded strategy. A **Correlation agent** links findings across specialists into compound risks and flags disagreements; a **Red-Team (Verification) agent** challenges every finding with its own checks and rejects what does not hold; a **Risk agent** scores only verified findings and a **Recommendation agent** writes fixes, with any GitHub action held for human approval. After the run, users can **ask the agents questions**: each question is routed to the agent responsible for that topic, which answers from its own evidence and cites the findings it used. On the bundled reference repository, Skopeo produced 24 findings, rejected 3 that did not hold up (including two "exposed" AWS keys that were documented example values), measured an N+1 query to settle a disagreement between two agents, recovered from an injected agent failure, and ranked a compound authentication-upgrade risk as P0.

Target users

Engineering leads and tech-debt owners deciding what to fix first; security reviewers triaging scanner output before a release or an acquisition; and maintainers evaluating an open-source dependency before adopting or upgrading it. Users sign in with Google or GitHub, import their repositories, follow the investigation live, and get a verified, explained risk report rather than a list of raw alerts.

Scope since the CA2 proposal

> The core of the approved project — a team of specialist agents that investigates a repository, cross-checks itself and scores verified risk — is unchanged. During CA3 we **extended** it to make it usable as a real, deployed product:
>
> - **Ask the agents**: per-case chat in which a router agent sends each question to the responsible agent (added to show agent responsibilities directly to users);
> - **Sign-in with Google or GitHub** and **import of the user's GitHub repositories** (so users need not paste URLs, and private repositories can be analysed with the owner's consent);
> - **Projects and upstream watches** with email/Slack/Discord/webhook notifications, showing which lines of the user's own code a change in a watched open-source dependency affects;
> - **Groq** as the production LLM provider and **PostgreSQL** support, so the system can be hosted for free without losing data.
>
> No feature was removed.